Decentralized Smart EV Charging

Xiangyu LUO

January 2026

Contents

1 Model and Game Analysis **2**
1.1 Merit-Order Formulation of the CO_2 Function 2
1.2 Individual EV Optimization 3
1.3 Water-Filling Optimality and Implementation 4
1.4 Potential Game Structure and BRD Update 4
1.5 Equilibria, Global Optimality, and Non-Uniqueness 5
1.6 Information Structure and Practical Interpretation 6

2 Experimental Setup **6**
2.1 Scenario and Policy Configuration 6
2.2 Evaluation Procedure 7

3 Numerical Results **7**
3.1 Convergence of BRD 7
3.2 Comparison with the Centralized Optimum 8
3.3 Emission Performance by Policy 9
3.4 Load-Shape Mechanisms Behind the Scores 10
3.5 Sensitivity to Nuclear Availability 12

4 Discussion and Realism **13**
4.1 Scaling and Mean-Field Interpretation 13
4.2 Practical Deployment of Policies 14
4.3 Limits of the Approximation 14

5 Conclusion **14**

A Merit-Order Context and CO_2 Function **16**

B Data Interface and Policy Defin. **17**

C Scoring and BRD Coordination **20**

1 Model and Game Analysis

We consider one day split into $T = 48$ half-hour slots ($\delta = 0.5$ h). For each EV $j \in \{1, \dots, J\}$, $\ell_j^{\mathrm{EV}}(t)$ is charging power (kW) at slot t. Let $\tilde{\ell}_{\mathrm{NF}}(t)$ be the non-flexible load forecast and define total load

$$L(t) = \tilde{\ell}_{\mathrm{NF}}(t) + \sum_{m=1}^{J} \ell_m^{\mathrm{EV}}(t).$$

The CO_2 objective uses the convex piecewise-linear function $f_{\mathrm{CO_2}}(\cdot)$ derived from the merit-order approximation in the reference document.

1.1 Merit-Order Formulation of the CO_2 Function

The CO_2 model follows a merit-order approximation in five steps: define effective capacities, sort assets by variable cost (tie-break by emission rate), build cumulative breakpoints, assign each load increment to its active segment, and aggregate segment emissions. Let P_i^{name} be the installed capacity of asset i and define effective capacities

$$P_i = \kappa_i P_i^{\mathrm{name}},$$

with $\kappa_i = 0.3$ for wind, $\kappa_i = 0.2$ for solar, $\kappa_i = \nu$ for nuclear (availability factor), and $\kappa_i = 1$ otherwise. After sorting, cumulative breakpoints are

$$\hat{P}_i := \sum_{k=1}^{i} P_k, \qquad \hat{P}_0 := 0.$$

For demand level ℓ (MW), the dispatched quantity on segment i is

$$q_i(\ell) := \min\{P_i,\, [\ell - \hat{P}_{i-1}]_+\} = \left[\min\{\ell, \hat{P}_i\} - \hat{P}_{i-1}\right]_+ .$$

Let r_i be combustion emission rates ($\mathrm{gCO_2eq/kWh}$) and define cumulative rates

$$\gamma_i := \sum_{k=1}^{i} r_k, \qquad \gamma_0 := 0.$$

The hourly emission score implemented in code is the piecewise-linear map

$$f_{\mathrm{CO_2}}(\ell) = 1000 \sum_{i=1}^{I} \gamma_i\, q_i(\ell), \tag{1}$$

where $[\cdot]_+ = \max(\cdot, 0)$. On interval $\ell \in [\hat{P}_{i-1}, \hat{P}_i]$, the slope is $1000\gamma_i$. Since all $r_i \geq 0$, the sequence (γ_i) is nondecreasing, which gives convexity. The factor 1000 converts MW to kW, so $f_{\mathrm{CO_2}}$ is in $\mathrm{gCO_2eq/h}$.

Equation (1) is a broken-line function with slope changes at breakpoints ($\hat{P}_i$). In code, assets are first sorted; then cumulative capacities (`p_hat`) and cumulative rates are built before segment-wise accumulation (Appendix A).

Daily emissions are then computed by time aggregation:

$$\mathrm{CO2}_{\mathrm{day}} = \sum_{t=1}^{48} f_{\mathrm{CO_2}}(L(t))\, \delta, \qquad \delta = 0.5 \text{ h}.$$

Capacity-factor and availability assumptions are therefore embedded directly in the breakpoints and are consistently reused in the sensitivity analysis.

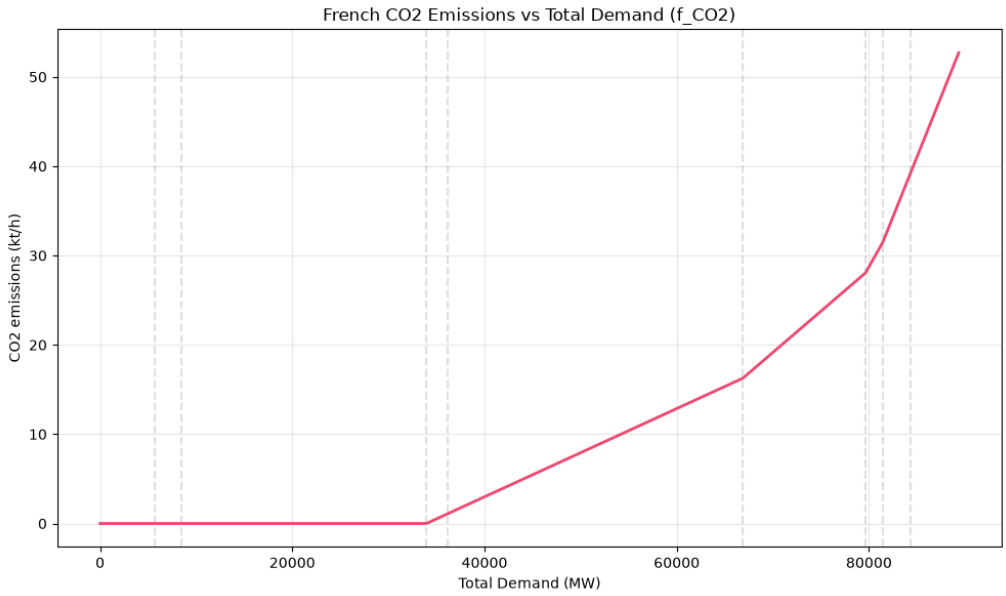


Figure 1: Piecewise-linear approximation of $f_{\mathrm{CO}_2}(L)$.

1.2 Individual EV Optimization

Given the profiles of other EVs, EV j solves

$$\mathcal{P}_j : \quad \min_{\ell_j^{\mathrm{EV}}} \sum_{t=1}^{T} f_{\mathrm{CO}_2}\left(\tilde{\ell}_{\mathrm{NF}}(t) + \sum_{m\neq j} \ell_m^{\mathrm{EV}}(t) + \ell_j^{\mathrm{EV}}(t)\right) \quad \text{s.t.} \quad \left(\ell_j^{\mathrm{EV}}(t)\right)_{t=1}^{T} \in \mathcal{L}_j. \tag{2}$$

The feasible set is

$$\mathcal{L}_j = \left\{ (\ell_j^{\mathrm{EV}}(t))_{t=1}^{T} : \begin{array}{l} \ell_j^{\mathrm{EV}}(t) = 0 \text{ for } t \notin \{a_j+1,\ldots,d_j-1\}, \\ 0 \le \ell_j^{\mathrm{EV}}(t) \le 7 \text{ for } t \in \{a_j+1,\ldots,d_j-1\}, \\ \sum_{t=a_j+1}^{d_j-1} \ell_j^{\mathrm{EV}}(t) = \dfrac{e_j}{\delta}. \end{array} \right\}. \tag{3}$$

Since f_{CO_2} is convex and the constraints are affine plus box bounds, each individual problem is convex (possibly non-smooth at breakpoints).

The EV objective is written in terms of system-level emissions, not only private charging convenience. In that sense, the player-level optimization is aligned with the global objective. This alignment yields favorable equilibrium properties, but it is also a normative modeling choice rather than a purely behavioral one.

At system scale, one vehicle has negligible unilateral influence: its maximum charging power (7 kW) is around 10^{-7} of a 60–70 GW national load level. Hence, in isolation, one EV cannot materially modify the emission trajectory. The relevance emerges only at fleet level, where coordinated responses produce non-negligible aggregate load shifts.

1.3 Water-Filling Optimality and Implementation

Fix

$$b(t) := \tilde{\ell}_{\mathrm{NF}}(t) + \sum_{m \neq j} \ell_m^{\mathrm{EV}}(t),$$

and define $x(t) := \ell_j^{\mathrm{EV}}(t)$ on $\mathcal{T}_j = \{a_j + 1, \ldots, d_j - 1\}$. Then

$$\min_x \sum_{t \in \mathcal{T}_j} f_{\mathrm{CO}_2}(b(t) + x(t)) \quad \text{s.t.} \quad 0 \leq x(t) \leq P_{\max}, \ \sum_{t \in \mathcal{T}_j} x(t) = E, \tag{4}$$

with $P_{\max} = 7$ kW and $E = e_j/\delta$.

At optimum $x^\star$, there exist $\lambda \in \mathbb{R}$ and nonnegative multipliers μ_t^-, μ_t^+ such that

$$0 \in \partial f_{\mathrm{CO}_2}(b(t) + x^\star(t)) + \lambda - \mu_t^- + \mu_t^+, \qquad \mu_t^- x^\star(t) = 0, \ \mu_t^+(P_{\max} - x^\star(t)) = 0. \tag{5}$$

Thus, interior active slots equalize marginal cost levels; with increasing slopes, energy must be allocated first on the currently lowest-load slots.

An equivalent exchange argument is immediate: if $L_1 < L_2$ with $x_2 > 0$ and $x_1 < P_{\max}$, shifting an infinitesimal charge from slot 2 to slot 1 does not increase objective value by convexity.

The implemented heuristic exactly follows this logic.

Algorithm 1 Water-Filling Best Response for One EV

Require: EV parameters (a_j, d_j, e_j), baseline load $b(t)$, increment ϵ, maximum iterations K
Ensure: Charging profile $x_j(t)$

Initialize $x_j(t) \leftarrow 0$ for all $t \in \{1, \ldots, 48\}$
$E \leftarrow e_j/\delta$, $k \leftarrow 0$
while $k < K$ **and** $\sum_t x_j(t) < E$ **do**
    $k \leftarrow k + 1$
    $\mathcal{T}_{\mathrm{nonsat}} \leftarrow \{t \in \{a_j + 1, \ldots, d_j - 1\} : x_j(t) < 7\}$
    if $\mathcal{T}_{\mathrm{nonsat}} = \varnothing$ **then**
        break
    end if
    $\mathcal{T}_{\min} \leftarrow \arg\min_{t \in \mathcal{T}_{\mathrm{nonsat}}} \big(b(t) + x_j(t)\big)$
    $\Delta \leftarrow \min\left(\epsilon, \dfrac{E - \sum_t x_j(t)}{|\mathcal{T}_{\min}|}\right)$
    for all $t \in \mathcal{T}_{\min}$ **do**
        $x_j(t) \leftarrow \min\{x_j(t) + \Delta, 7\}$
    end for
end while

1.4 Potential Game Structure and BRD Update

Define the social potential

$$\Phi(\ell_1^{\mathrm{EV}}, \ldots, \ell_J^{\mathrm{EV}}) := \sum_{t=1}^{T} f_{\mathrm{CO}_2}\left(\tilde{\ell}_{\mathrm{NF}}(t) + \sum_{m=1}^{J} \ell_m^{\mathrm{EV}}(t)\right). \tag{6}$$

For fixed ℓ_{-j}, player j minimizes $\Phi(\ell_j, \ell_{-j})$. The game is therefore an exact potential game and, more specifically, a team game.

The BRD procedure implemented in this work is reported below.

Algorithm 2 Best-Response Dynamics (BRD)

Require: EV set $\{1, \dots, J\}$, non-flexible load $L_{\mathrm{nf}}(t)$, fleet scaling factor α, threshold η, max iterations K

Ensure: Charging profiles $p_j(t)$

Initialize $p_j(t) = 0$ for all j, t; initialize aggregate EV load $L_{\mathrm{EV}}(t) = 0$
for $k = 1$ to K **do**
  Store previous profiles $p_j^{\mathrm{prev}}(t)$
  for all $j \in \{1, \dots, J\}$ **do**
    $L_{\mathrm{base}}(t) \leftarrow L_{\mathrm{nf}}(t) + \alpha \cdot \big(L_{\mathrm{EV}}(t) - p_j(t)\big)$
    $p_j(t) \leftarrow$ Water-Filling best response of EV j under $L_{\mathrm{base}}(t)$
    $L_{\mathrm{EV}}(t) \leftarrow L_{\mathrm{EV}}(t) - p_j^{\mathrm{prev}}(t) + p_j(t)$
  end for
  diff $\leftarrow \sum_{j=1}^{J} \|p_j - p_j^{\mathrm{prev}}\|_2^2$
  if diff $< \eta$ **then**
    break
  end if
end for

Each block update cannot increase Φ, so BRD is a cyclic block-coordinate descent on a convex potential.

1.5 Equilibria, Global Optimality, and Non-Uniqueness

Let $\mathcal{L} = \mathcal{L}_1 \times \cdots \times \mathcal{L}_J$, $\mathrm{OPT} = \min_{\ell \in \mathcal{L}} \Phi(\ell)$, and write $L(t) = \tilde{\ell}_{\mathrm{NF}}(t) + \sum_m \ell_m^{\mathrm{EV}}(t)$ for the total load. Existence of a Nash equilibrium follows from continuity of Φ and compactness of $\mathcal{L}$: every minimizer of Φ over $\mathcal{L}$ is an equilibrium.

Nonsmooth potentials: equilibria need not be optimal. If Φ were convex and differentiable, the blockwise optimality conditions $0 \in \nabla_j \Phi(\ell^\star) + N_{\mathcal{L}_j}(\ell_j^\star)$ would stack into $0 \in \nabla\Phi(\ell^\star) + N_{\mathcal{L}}(\ell^\star)$, and every equilibrium would be globally optimal. For a nonsmooth convex potential this stacking step fails, because each block may use a different element of the subdifferential. A two-player example with identical costs $\Phi(x, y) = \max\{2-x,\, 2-y\}$ on $[0, 1]^2$ shows this: no unilateral deviation from $(0, 0)$ lowers $\Phi(0, 0) = 2$, yet $\Phi(1, 1) = 1$. Since $f_{\mathrm{CO_2}}$ is piecewise linear, we therefore do not claim that every equilibrium of the $\mathrm{CO_2}$ game is optimal. The profiles computed by BRD have additional structure, which is enough.

Valley-filling profiles are globally optimal. Call $\ell^\star \in \mathcal{L}$ *valley-filling* if, for every EV j and all $s, t \in \mathcal{T}_j$,

$$\ell_j^\star(s) > 0 \text{ and } \ell_j^\star(t) < P_{\max} \quad \Longrightarrow \quad L^\star(s) \le L^\star(t). \tag{7}$$

This is the condition enforced by Algorithm 1: each EV charges only in the lowest-load slots it can still use. It does not refer to $f_{\mathrm{CO_2}}$.

Proposition 1 (Valley-filling equilibria are optimal)**.** *Let f be any convex function and $\Phi(\ell) = \sum_t f(L(t))$. If $\ell^\star \in \mathcal{L}$ is valley-filling, then $\Phi(\ell^\star) = \mathrm{OPT}$. In particular, $\ell^\star$ is a Nash equilibrium whose value is optimal for every convex emission curve.*

Proof. Let $h(u)$ be the right derivative of f at u; it is nondecreasing and $h(u) \in \partial f(u)$. Put $g_t := h(L^\star(t))$. For each j, consider the linear problem $\min_{x \in \mathcal{L}_j} \sum_{t \in \mathcal{T}_j} g_t x(t)$. A point of $\mathcal{L}_j$ (box constraints and one sum constraint) is optimal for this problem if and only if no exchange is profitable, that is, there are no s, t with $x(s) > 0$, $x(t) < P_{\max}$ and $g_t < g_s$. By (7) and monotonicity of h, $\ell_j^\star$ has no such pair, so it minimizes the linear cost. For any $\ell \in \mathcal{L}$, the subgradient inequality in each slot gives

$$\Phi(\ell) \;\geq\; \Phi(\ell^\star) + \sum_t g_t\big(L(t) - L^\star(t)\big) = \Phi(\ell^\star) + \sum_j \sum_{t \in \mathcal{T}_j} g_t\big(\ell_j(t) - \ell_j^\star(t)\big) \;\geq\; \Phi(\ell^\star).$$

□

The key point is that all EVs use the *same* subgradient g_t in a slot, because the valley-filling rule ranks slots by load rather than by an arbitrary subgradient of f_{CO_2}. The implemented routine fills in increments ϵ, and BRD stops when the move norm falls below η, so the computed profiles satisfy (7) up to these tolerances. Section 3.2 compares them with the centralized optimum.

Uniqueness of the optimal value is guaranteed, but uniqueness of schedules is not, because f_{CO_2} is piecewise linear and Φ is generally not strictly convex. BRD may converge to different profiles with the same final CO_2 score.

1.6 Information Structure and Practical Interpretation

The optimization of EV j depends on

$$b_j(t) = \tilde{\ell}_{\mathrm{NF}}(t) + \sum_{m \neq j} \ell_m^{\mathrm{EV}}(t),$$

which is an aggregate quantity. Therefore, the mechanism does not require each EV to observe all individual neighbor schedules. This aggregate-only dependency is consistent with a privacy-preserving decentralized implementation.

However, practical deployment still requires communication infrastructure and trusted signal generation. In operational terms, an aggregator (or system operator) must broadcast a reliable aggregate baseline or equivalent marginal signal; agents then solve local updates against this signal. Forecast error and delayed information can affect convergence speed and final quality, even if the static model predicts strong optimality properties.

2 Experimental Setup

2.1 Scenario and Policy Configuration

The experiments cover summer/winter conditions in 2019 and 2020. A simulated set of $J = 500$ EVs is scaled to a fleet of $N = 15$ million EVs. The charging policies are summarized in Table 1.

Table 1: Charging policies used in the experiments.

Policy	Implementation rule
Plug-and-Charge (PC)	Charging starts immediately after arrival, up to 7 kW, until energy need is met.
Off-Peak (OP)	Start time is drawn uniformly (seeded) in the off-peak window (22:00 to 06:00), then charging proceeds at up to 7 kW.
Off-Peak, synchronized (OP-22)	Every EV starts at 22:00, the literal off-peak tariff signal, at up to 7 kW. Used as a herding stress test in the load-shape and threshold diagnostics.
Smart Charging (BRD)	Sequential best-response updates with Water-Filling as per-EV optimizer.

2.2 Evaluation Procedure

The global simulation workflow is the following.

Algorithm 3 Policy Evaluation Workflow

Require: Scenario (y, s), EV scenario set, baseline load $L_{\text{nf}}(t)$, fleet size N, simulation size N_{sim}
Ensure: Policy-level emission scores and load curves
1: Load $L_{\text{nf}}(t)$ and EV arrivals/departures/energy needs
2: Set scaling factor $\alpha \leftarrow N/N_{\text{sim}}$
3: Construct merit-order CO_2 model f_{CO_2}
4: **for all** $P \in \{\text{PC, OP, BRD}\}$ **do**
5: Compute individual charging profiles under policy P
6: Aggregate EV load and total load
7: Evaluate total emissions $\sum_t f_{CO_2}(L^P_{\text{tot}}(t))\,\delta$
8: **end for**
9: Export aggregated indicators and figures

3 Numerical Results

3.1 Convergence of BRD

Table 2 shows that BRD converges in 3 to 7 sweeps depending on scenario, with small final move norms. This confirms that most of the useful adjustment happens in early iterations. The scenario dependence is meaningful: cases with flatter low-load periods tend to require more redistribution steps before the stopping criterion is reached.

Figure 2 provides the dynamic view. The first sweeps produce the steepest drop, indicating rapid elimination of inefficient charging placements. The tail of the curve is much flatter, which is expected under a convex but non-strictly convex potential: late updates mainly reallocate charging among nearly equivalent slots with very small objective improvements.

Table 2: BRD stopping statistics by scenario.

Scenario	BRD iterations	Final move norm
2019 Summer	5	0.145083
2019 Winter	3	0.618738
2020 Summer	7	0.718181
2020 Winter	4	0.117583

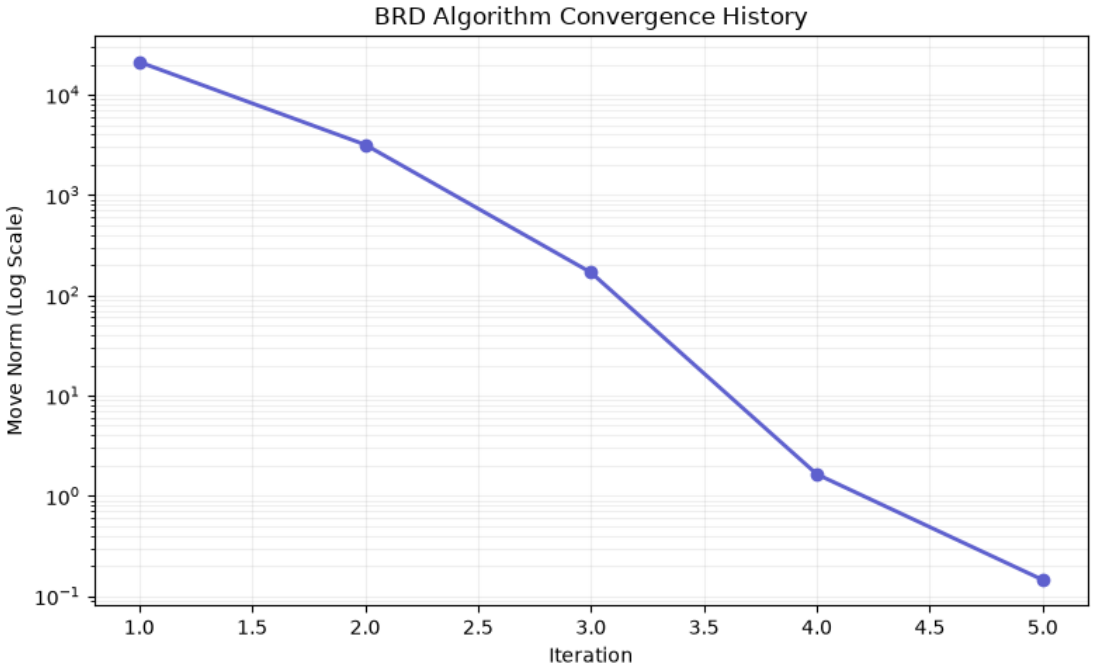


Figure 2: BRD convergence trajectory.

3.2 Comparison with the Centralized Optimum

Because f_{CO_2} is convex and piecewise linear, the centralized problem $\min_{\ell \in \mathcal{L}} \Phi(\ell)$ is a linear program: one epigraph variable per slot, one inequality per slot and merit-order segment, and the individual window, power and energy constraints of the 500 simulated EVs (scaled to the fleet as in BRD). The script `centralized_check.py` solves it with HiGHS through `scipy.optimize.linprog` and evaluates the BRD profiles with the same scoring function.

Table 3: BRD emissions versus the centralized LP optimum (daily CO_2, tonnes).

Scenario	BRD sweeps	BRD	LP optimum	Relative gap
2019 Summer	5	141,684	141,684	$< 10^{-15}$
2019 Winter	3	332,257	332,257	$< 10^{-15}$
2020 Summer	7	85,640	85,640	$< 10^{-15}$
2020 Winter	4	399,541	399,541	$< 10^{-15}$

In all four scenarios, BRD attains the LP optimum up to floating-point rounding, as predicted

by Proposition 1. The decentralized mechanism therefore loses nothing relative to a central planner with full information in these cases.

3.3 Emission Performance by Policy

Table 4 reports absolute daily emissions and relative gains in a single view. Three findings emerge.

First, BRD is never worse than the alternatives and is strictly better than PC in every scenario. The reduction reaches 16.03% in 2019 winter and 15.16% in 2020 winter.

Second, OP is beneficial compared with PC but systematically dominated by BRD in winter and 2020 summer. The BRD-over-OP gain is substantial in winter: about 19,593 tons/day in 2019 winter and 19,946 tons/day in 2020 winter. In 2020 summer the gain is smaller (about 3,202 tons/day), consistent with a lower average marginal carbon intensity.

Third, 2019 summer is a boundary case where OP and BRD are numerically identical at the reported precision. This indicates that in that low-stress regime, a simple off-peak shift can already place most EV energy on low-carbon merit-order segments, leaving little room for additional optimization.

Table 4: Policy-level CO_2 performance by scenario.

Year	Season	PC (t)	OP (t)	BRD (t)	OP vs. PC	BRD vs. PC	BRD vs. OP
2019	Summer	143,790	141,684	141,684	1.46%	1.46%	0.00%
2019	Winter	395,700	351,850	332,257	11.08%	16.03%	5.57%
2020	Summer	93,699	88,842	85,640	5.18%	8.60%	3.60%
2020	Winter	470,941	419,486	399,541	10.93%	15.16%	4.75%

Figure 3 confirms these cross-scenario patterns visually: inter-policy spacing is narrow in summer, while winter bars are clearly separated, with BRD lowest.

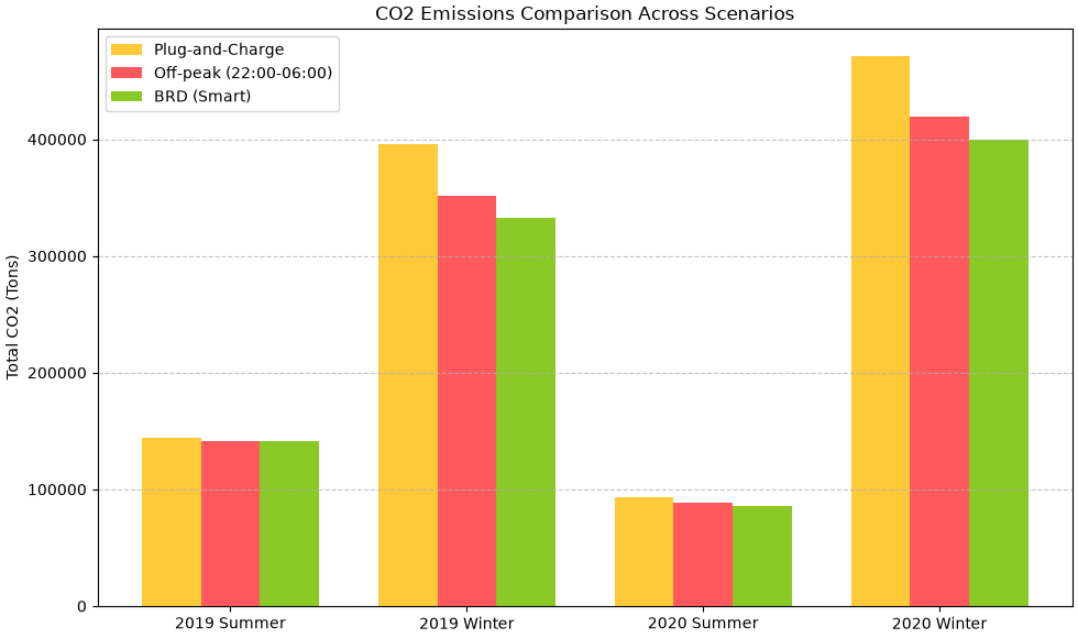
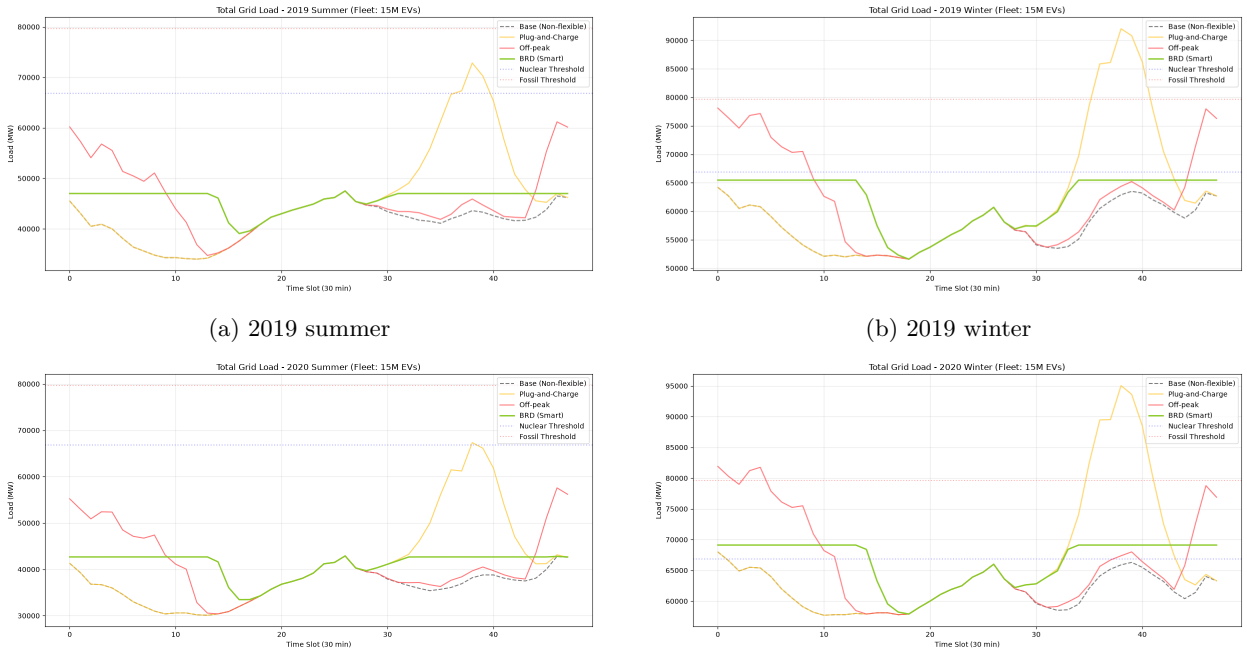


Figure 3: Total CO_2 emissions by policy and scenario.

3.4 Load-Shape Mechanisms Behind the Scores

To align figures with the argument, we first analyze total-load trajectories, then EV-only profiles, and finally threshold exceedance metrics.

Figure 4 shows total load for the four scenarios. Two systematic effects appear. PC shifts substantial energy toward early charging hours and raises evening-night peaks. OP displaces this peak into the night window; since start times are spread over eight hours its peak is lower than under PC, but charging is still placed by the clock rather than by the load level, so it piles up on the night hours regardless of where the valleys are. BRD produces the flattest trajectories and keeps the system near the low-to-intermediate load bands for longer periods.



(a) 2019 summer

(b) 2019 winter

(c) 2020 summer

(d) 2020 winter

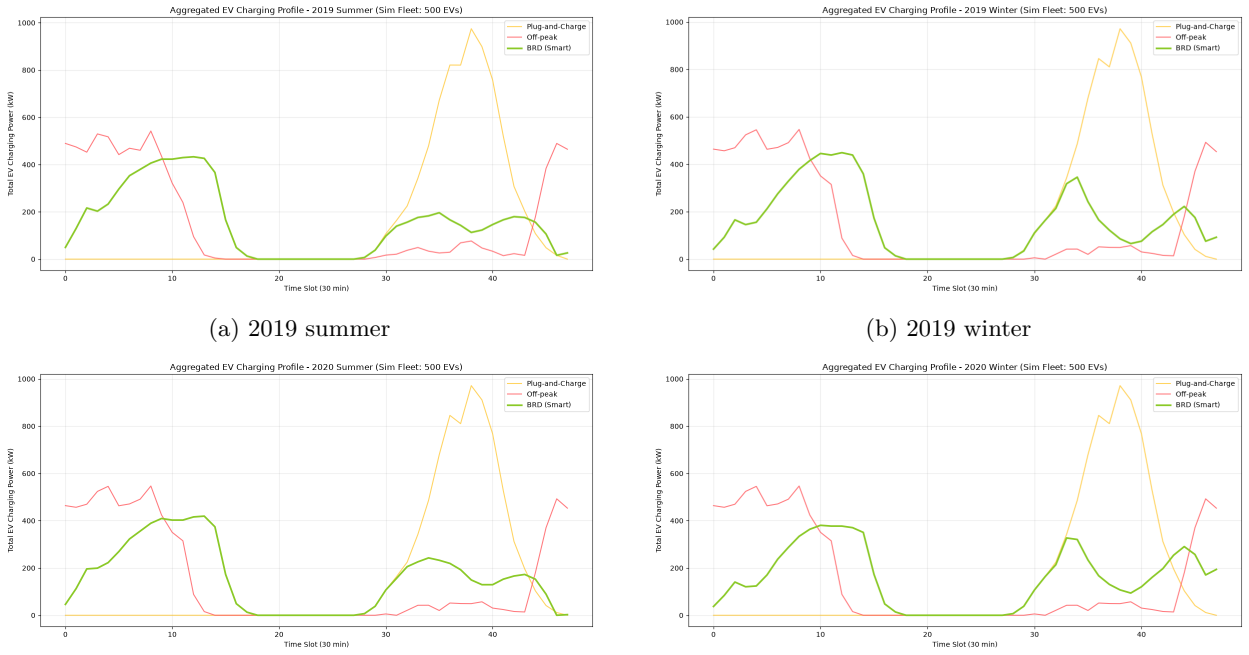
Figure 4: Total grid load under PC, OP, and BRD across all scenarios.

Table 5 quantifies this visual pattern. PC raises the base load peak by +40% to +57%. Randomized OP reduces this to +21% to +34%, whereas the synchronized variant OP-22, in which the whole fleet reacts to the same 22:00 signal, produces by far the largest peaks (+143% to +234%). BRD keeps peak growth near zero (0% in both summer cases, about +2% in winter). This is the central operational difference between coordinated valley filling and clock-synchronized charging.

Table 5: Peak-load impact by scenario.

Scenario	Base peak (MW)	PC peak (MW)	OP peak (MW)	OP-22 peak (MW)	BRD peak (MW)	BRD vs. base
2019 Summer	47,500	72,844	61,188	147,300	47,500	0.00%
2019 Winter	64,200	92,042	78,114	163,800	65,468	1.98%
2020 Summer	42,900	67,342	57,578	143,100	42,900	0.00%
2020 Winter	68,000	95,042	81,914	165,400	69,118	1.64%

Figure 5 isolates EV demand. PC is front-loaded after arrivals; OP concentrates power around the off-peak window; BRD spreads charging over a wider feasible support. This explains why BRD lowers carbon cost even with identical total EV energy: it reallocates charging across time rather than reducing energy volume.



(a) 2019 summer

(b) 2019 winter

(c) 2020 summer

(d) 2020 winter

Figure 5: EV-only aggregate charging profiles.

Figure 6 provides a focused illustration of the BRD mechanism in winter. The optimized curve fills low-demand valleys first and creates a broad plateau rather than a narrow peak, which is precisely the geometry predicted by the KKT marginal-equalization argument in Section 1.3.

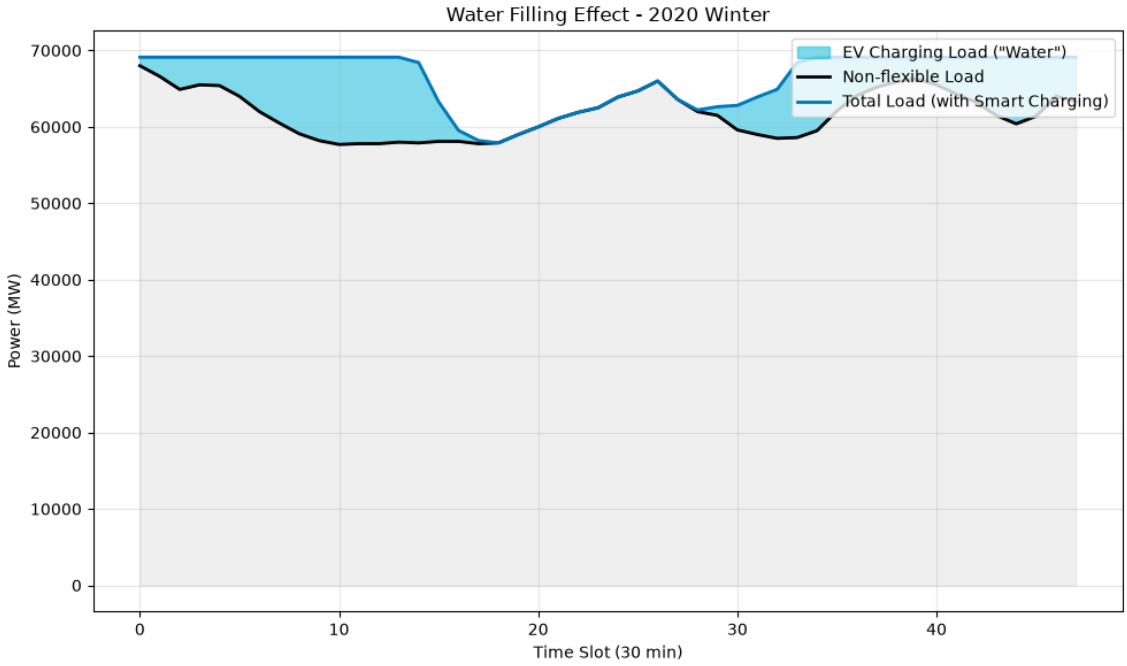


Figure 6: Water-filling effect in 2020 winter.

Figure 7 details the most constrained case (2020 winter). PC reaches the highest instantaneous

peak (about 95 GW). OP reduces this specific peak (about 82 GW) but still crosses the gas threshold in several slots. BRD keeps the trajectory closer to the nuclear threshold and avoids gas-threshold crossing.

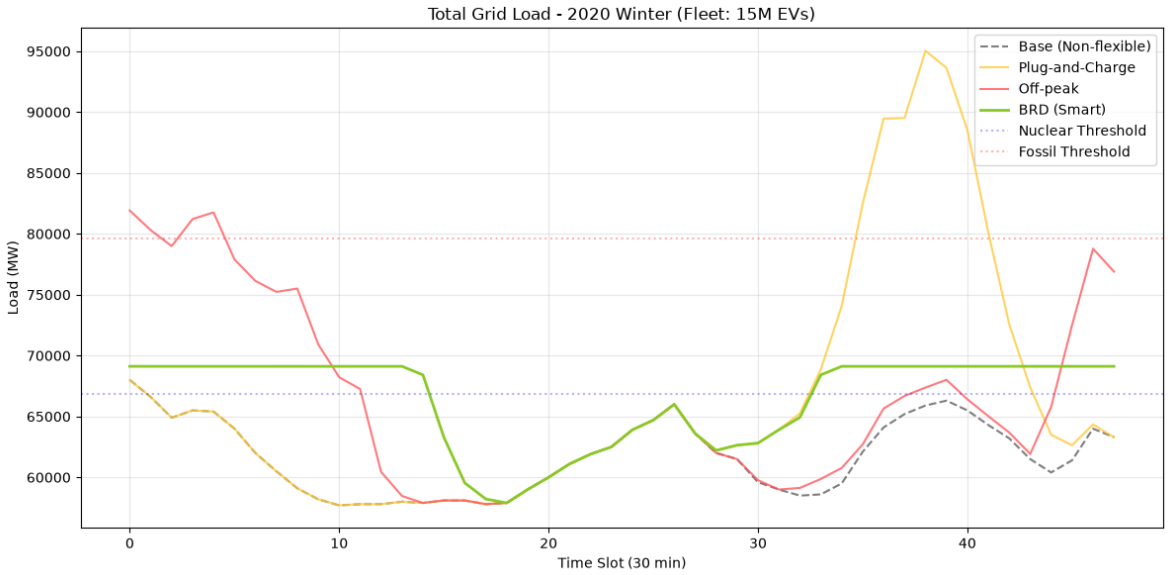


Figure 7: Detailed total-load comparison for 2020 winter.

Table 6 confirms this across scenarios. The synchronized OP-22 policy exceeds the gas threshold in every scenario, randomized OP does so in 2020 winter, and BRD keeps gas exceedance at zero in all four scenarios. For nuclear-threshold exceedance, 2020 winter is instructive: BRD has more hours above threshold but much lower exceedance severity (32,935 MW-hours, against 71,028 for OP and 104,406 for OP-22), indicating flatter near-threshold operation instead of extreme spikes.

Table 6: Threshold exceedance indicators.

Scenario	**OP** $h > g$	**OP-22** $h > g$	**BRD** $h > g$	**OP MW-h**$> n$	**OP-22 MW-h**$> n$	**BRD MW-h**$> n$
2019 Summer	0.0	1.0	0.0	0	70,909	0
2019 Winter	0.0	1.5	0.0	45,672	100,156	0
2020 Summer	0.0	1.0	0.0	0	65,428	0
2020 Winter	2.0	1.5	0.0	71,028	104,406	32,935

3.5 Sensitivity to Nuclear Availability

Figure 8 shows a monotone decrease of BRD gain versus PC as nuclear availability increases. Quantitatively, the gain drops from about 21.67% at availability 0.3 to 16.03% at 0.5, then 7.66% at 0.6, and reaches approximately zero for 0.9–1.0.

The interpretation is direct in merit-order terms: with lower nuclear availability, the system hits higher-emission marginal technologies more often, so smart demand shifting avoids expensive carbon segments and yields large benefits. As nuclear availability rises, marginal emission rates remain lower over a wider load range, so shifting demand has weaker incremental effect.

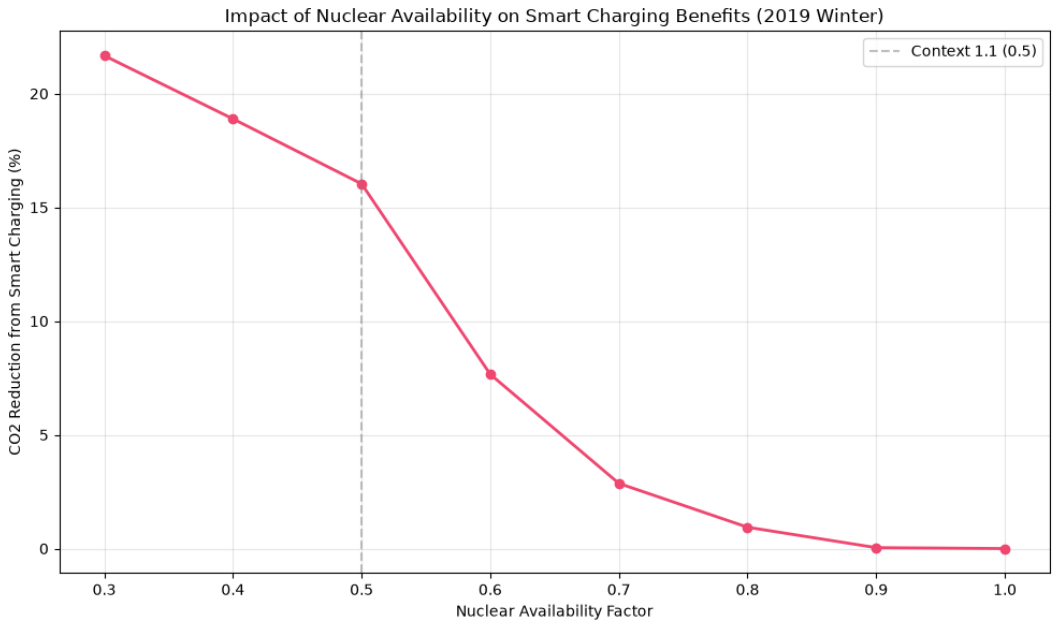


Figure 8: Impact of nuclear availability on BRD relative gains (2019 winter).

4 Discussion and Realism

4.1 Scaling and Mean-Field Interpretation

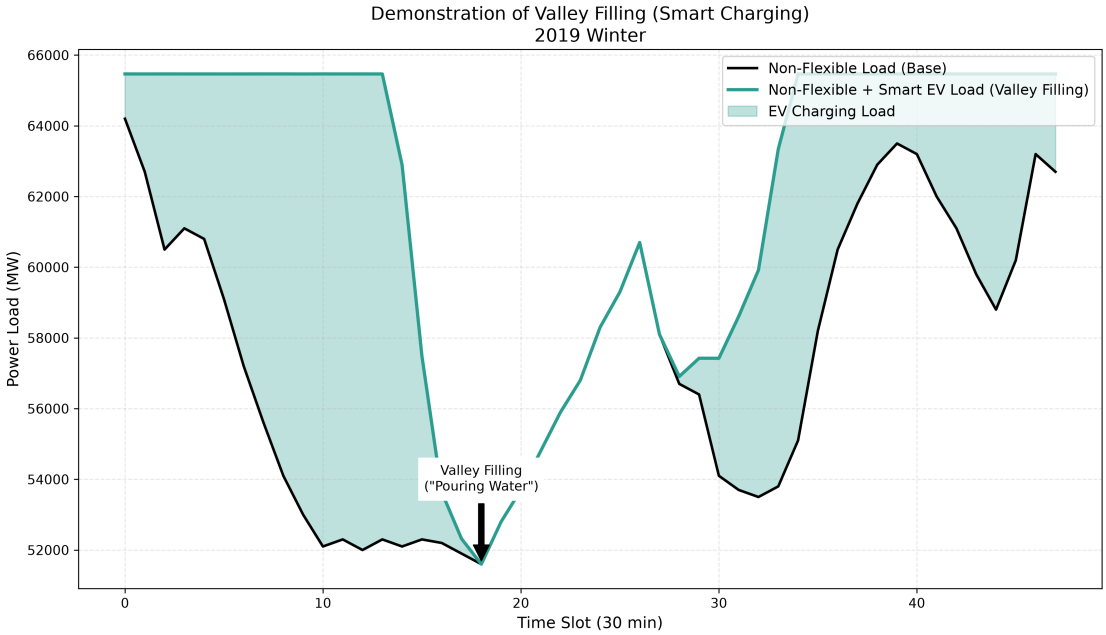


Figure 9: Mean-field valley-filling interpretation.

The national profile is obtained from the simulated profile through

$$\ell^{\text{national}}(t) = \frac{15{,}000{,}000}{500}\,\ell^{\text{sim}}(t).$$

This scaling is justified by a mean-field view: each EV has negligible unilateral impact and reacts to aggregate signals. Figure 9 illustrates how decentralized best responses create a collective valley-filling pattern even though each agent solves only a local constrained problem.

4.2 Practical Deployment of Policies

Plug-and-Charge is the easiest to deploy: no communication, no forecasting, no coordination logic. Its limitation is structural: it amplifies local temporal synchronization around arrival times and yields higher system peaks and emissions.

Off-Peak tariff control is also simple to communicate and can be rolled out through existing tariff instruments. However, fixed clock-time incentives create herding behavior: many EVs react to the same temporal signal, so demand concentration is displaced rather than removed. The rebound behavior observed in results, most visible in the synchronized OP-22 variant, is a direct consequence of this synchronized response.

BRD-based smart charging has higher implementation requirements. It needs periodic aggregate signals, iterative updates, and digital control capabilities at charging endpoints. In return, it avoids hard synchronization, uses flexibility windows more efficiently, and delivers lower CO_2 scores together with improved peak control. In this sense, BRD trades implementation complexity for robustness of system-level outcomes.

4.3 Limits of the Approximation

The model is intentionally simplified. Real dispatch follows Unit Commitment with binary on/off constraints, startup costs, minimum up/down times, reserve constraints, and ramping limits. Hence the convex piecewise-linear f_{CO_2} is a tractable approximation rather than a full operational simulator. The results should be interpreted as structurally informative: they identify robust directional effects (benefit of coordination, rebound risk of synchronized policies, sensitivity to generation mix) rather than exact day-ahead operational forecasts.

5 Conclusion

This report addresses the main study questions with a unified theoretical and numerical analysis.

1. Under the convex piecewise-linear emission model and Cartesian feasibility sets, the decentralized game is an exact potential team game. Because the emission curve is nonsmooth, not every equilibrium need be optimal; valley-filling equilibria, which water-filling best responses produce, are globally optimal for any convex emission curve (Proposition 1), and BRD matches the centralized LP optimum in all four scenarios. Charging profiles may be non-unique.

2. The single-EV optimization admits a Water-Filling solution structure, and the BRD coordination loop converges quickly in the tested cases (3 to 7 sweeps).

3. Across the four evaluated scenarios, BRD is never worse than the baselines in CO_2 score; gains are largest in winter conditions, and OP and BRD are equal at reported precision in 2019 summer.

4. Load-shape and threshold analyses show that BRD reduces synchronization effects, keeps peak amplification low, and avoids gas-threshold exceedance in the reported scenarios.

5. Sensitivity results confirm that smart-charging gains decrease as nuclear availability increases. The results remain an approximation based on merit-order piecewise modeling rather than a full unit-commitment simulator.

A Merit-Order Context and CO_2 Function

Listing 1: Model context and piecewise-linear CO_2 evaluation (`smart_charging.py`, lines 20–86).

```
€
# Capacity factors suggested in the TP statement to account for intermittency.
WIND_CAPACITY_FACTOR = 0.3
SOLAR_CAPACITY_FACTOR = 0.2
def get_model_context(nuclear_availability=0.5):

    # Define assets
    assets = [
        {"name": "Wind", "capacity": 19099 * WIND_CAPACITY_FACTOR, "variable_cost": 0,
            "co2_rate": 0},
        {"name": "Solar", "capacity": 13612 * SOLAR_CAPACITY_FACTOR, "variable_cost":
            0, "co2_rate": 0},
        {"name": "Hydro pump.", "capacity": 25504, "variable_cost": 0, "co2_rate": 0},
        {"name": "Bioenergy", "capacity": 2234, "variable_cost": 0, "co2_rate": 494},
        {"name": "Nuclear", "capacity": 61370 * nuclear_availability, "variable_cost":
            30, "co2_rate": 0},
        {"name": "Gas", "capacity": 12752, "variable_cost": 70, "co2_rate": 429},
        {"name": "Coal", "capacity": 1818, "variable_cost": 86, "co2_rate": 986},
        {"name": "Fuel", "capacity": 2890, "variable_cost": 162, "co2_rate": 777},
    ]

    # Sort assets
    assets.sort(key=lambda a: (a["variable_cost"], a["co2_rate"]))

    # Calculate cumulative capacities P_hat
    p_hat = [0]
    for asset in assets:
        p_hat.append(p_hat[-1] + asset["capacity"])

    # Calculate Cumulative CO2 Rates
    cumulative_rates = []
    current_rate = 0
    for asset in assets:
        current_rate += asset["co2_rate"]
        cumulative_rates.append(current_rate)

    return {
        "assets": assets,
        "p_hat": p_hat,
        "cumulative_rates": cumulative_rates,
        "f_co2": lambda load: _f_co2_impl(load, assets, p_hat, cumulative_rates),
        "cumulative_capacity_until": lambda name: _cumulative_capacity_until_impl(name
            , assets)
    }

def _cumulative_capacity_until_impl(asset_name, assets):
    total = 0.0
    for asset in assets:
        total += asset["capacity"]
        if asset["name"] == asset_name:
            return total
    return None

def _f_co2_impl(load, assets, p_hat, cumulative_rates):
```

```
    """Calculates total CO2 emissions (g/h) for a given total load (MW) using
        cumulative rates."""
    total_co2 = 0
    remaining_load = load

    # Iterate through assets in merit order
    for i, asset in enumerate(assets):
        capacity = asset["capacity"]
        rate = cumulative_rates[i]

        allocated_load = min(remaining_load, capacity)
        if allocated_load > 0:
            total_co2 += allocated_load * 1000 * rate
            remaining_load -= allocated_load

        if remaining_load <= 1e-9:
            break

    # If load exceeds total capacity, apply last asset's cumulative rate to excess
```

B Data Interface and Policy Defin.

Listing 2: Scenario loading and policy definitions (`smart_charging.py`, lines 134–291).

```
€

    return day_label, np.array(nf_load)

def load_data(year=2019, season="summer", day=None, ev_limit=DEFAULT_SIM_FLEET_SIZE):
    """Loads EV scenarios + non-flexible load curve for a single day (48 slots)."""
    filename = f"data/eCO2mix_RTE_Annuel-Definitif_{year}_{season}.csv"
    if day is None:
        day_label, nf_load = _load_ec02mix_day_curve(filename, day=None)
    else:
        try:
            day_label = datetime.datetime.strptime(day, "%d/%m/%Y").strftime("%d/%m/%Y
                ")
        except ValueError:
            raise ValueError("day must be formatted as dd/mm/YYYY, e.g. 01/06/2019")
                from None
        _, nf_load = _load_ec02mix_day_curve(filename, day=day_label)

    if nf_load.size == 0:
        return [], np.array([])

    ev_day_label = day_label
    ev_rows = []
    with open("data/ev_scenarios.csv", "r") as f:
        reader = csv.DictReader(f, delimiter=";")
        for row in reader:
            if ev_day_label is None or row.get("day") == ev_day_label:
                ev_rows.append(row)
            if len(ev_rows) >= ev_limit:
                break

    if len(ev_rows) < ev_limit:
        with open("data/ev_scenarios.csv", "r") as f:
```

```
            reader = csv.DictReader(f, delimiter=";")
            for row in reader:
                if len(ev_rows) >= ev_limit:
                    break
                if ev_day_label is not None and row.get("day") == ev_day_label:
                    continue
                ev_rows.append(row)

    return ev_rows, nf_load

def run_scenario(year, season, day=None, ev_count=DEFAULT_SIM_FLEET_SIZE, fleet_size=FLEET_SIZE):
    ev_data, nf_load = load_data(year=year, season=season, day=day, ev_limit=ev_count)
    if not ev_data or nf_load.size == 0:
        return None

    test_evs = ev_data[:ev_count]
    sim_fleet_size = len(test_evs)
    fleet_scale = fleet_size / max(len(test_evs), 1)

    # Create CO2 model context for this scenario
    ctx = get_model_context()

    # 1. Plug-and-Charge
    pc = [policy_plug_and_charge(r) for r in test_evs]
    pc_co2 = calculate_co2_scores(nf_load, pc, fleet_scale, ctx)

    # 2. Off-peak (seeded start times)
    op_rng = np.random.default_rng(OFF_PEAK_SEED)
    op = [policy_off_peak(r, op_rng) for r in test_evs]
    op_co2 = calculate_co2_scores(nf_load, op, fleet_scale, ctx)

    # 3. BRD
    print(f"\n--- Running BRD for {year} {season} ---")
    brd, history = brd_coordination(test_evs, nf_load, fleet_scale=fleet_scale, ctx=ctx, max_k=10)
    brd_co2 = calculate_co2_scores(nf_load, brd, fleet_scale, ctx)

    # Plot load curves for this scenario
    plot_load_curves(
        nf_load,
        pc,
        op,
        brd,
        year,
        season,
        fleet_scale,
        fleet_size,
        f"results/load_{year}_{season}.png",
        ctx=ctx
    )

    # Plot EV-only comparison
    plot_ev_load_comparison(pc, op, brd, year, season, len(test_evs), f"results/ev_only_{year}_{season}.png")

    # Water-filling view of the BRD equilibrium
    plot_water_filling(nf_load, brd, fleet_scale, year, season, f"results/water_filling_{year}_{season}.png")
```

```
87
88      # Load-shape, emission and threshold indicators for every policy
89      op22 = [policy_off_peak_22(r) for r in test_evs]
90      policy_profiles = {"base": None, "plug_and_charge": pc, "off_peak": op, "
            off_peak_22": op22, "brd": brd}
91      scenario_metrics = [
92          policy_metrics(year, season, name, nf_load, profiles, fleet_scale, fleet_size,
                len(test_evs), ctx,
93                         brd_history=history if name == "brd" else None)
94          for name, profiles in policy_profiles.items()
95      ]
96
97      # Save first convergence history
98      if not os.path.exists("results/brd_convergence.png"):
99          plot_convergence(history)
100
101     return {
102         "year": year,
103         "season": season,
104         "sim_fleet_size": sim_fleet_size,
105         "fleet_size": fleet_size,
106         "pc_co2": pc_co2,
107         "op_co2": op_co2,
108         "brd_co2": brd_co2,
109         "metrics": scenario_metrics,
110     }
111
112 def total_load_mw(nf_load, profiles, fleet_scale):
113     """Non-flexible load plus the scaled EV load (MW)."""
114     if profiles is None or len(profiles) == 0:
115         return np.asarray(nf_load, dtype=float)
116     return nf_load + (np.sum(profiles, axis=0) * fleet_scale) / 1000
117
118 def policy_metrics(year, season, policy, nf_load, profiles, fleet_scale, fleet_size,
        sim_fleet_size, ctx,
119                    brd_history=None):
120     """Emission, load-shape and merit-order threshold indicators of one policy."""
121     base_load = np.asarray(nf_load, dtype=float)
122     load = total_load_mw(nf_load, profiles, fleet_scale)
123     ev_load = load - base_load
124     base_co2 = f_co2_total(base_load, ctx)
125     total_co2 = f_co2_total(load, ctx)
126     ev_energy_kwh = float(np.sum(ev_load) * SLOT_HOURS * 1000)
127     nuclear_thr = ctx["cumulative_capacity_until"]("Nuclear")
128     gas_thr = ctx["cumulative_capacity_until"]("Gas")
129     row = {
130         "year": year,
131         "season": season,
132         "policy": policy,
133         "J": sim_fleet_size,
134         "fleet_size": fleet_size,
135         "total_co2_tons": total_co2 / 1e6,
136         "ev_increment_co2_tons": (total_co2 - base_co2) / 1e6,
137         "ev_energy_gwh": ev_energy_kwh / 1e6,
138         "ev_intensity_g_per_kwh": (total_co2 - base_co2) / ev_energy_kwh if
                ev_energy_kwh > 0 else 0.0,
139         "total_load_peak_mw": float(np.max(load)),
140         "total_load_mean_mw": float(np.mean(load)),
141         "total_load_std_mw": float(np.std(load)),
```

```
        "ev_load_peak_mw": float(np.max(ev_load)),
        "base_peak_mw": float(np.max(base_load)),
        "base_mean_mw": float(np.mean(base_load)),
        "base_std_mw": float(np.std(base_load)),
        "brd_iters": len(brd_history) if brd_history is not None else "",
        "brd_final_move_norm": float(brd_history[-1]) if brd_history else "",
        "nuclear_threshold_mw": nuclear_thr,
        "gas_threshold_mw": gas_thr,
        "hours_above_nuclear": float(np.sum(load > nuclear_thr) * SLOT_HOURS),
        "hours_above_gas": float(np.sum(load > gas_thr) * SLOT_HOURS),
        "mw_hours_above_nuclear": float(np.sum(np.maximum(load - nuclear_thr, 0.0)) *
            SLOT_HOURS),
        "mw_hours_above_gas": float(np.sum(np.maximum(load - gas_thr, 0.0)) *
            SLOT_HOURS),
    }
    return row

INTERMEDIATE_FIELDS = [
    "year", "season", "policy", "J", "fleet_size", "total_co2_tons", "
        ev_increment_co2_tons", "ev_energy_gwh",
    "ev_intensity_g_per_kwh", "total_load_peak_mw", "total_load_mean_mw", "
        total_load_std_mw", "ev_load_peak_mw",
```

C Scoring and BRD Coordination

Listing 3: CO_2 scoring and BRD loop (`smart_charging.py`, lines 502–539).

```
€
    plt.fill_between(slots, 0, nf_load, color="lightgray", alpha=0.35)
    plt.fill_between(slots, nf_load, total, color="#00b4d8", alpha=0.5, label='EV
        Charging Load ("Water")')
    plt.plot(slots, nf_load, color="black", linewidth=2, label="Non-flexible Load")
    plt.plot(slots, total, color="#0077b6", linewidth=2, label="Total Load (with Smart
         Charging)")
    plt.title(f"Water Filling Effect - {year} {season.capitalize()}")
    plt.xlabel("Time Slot (30 min)")
    plt.ylabel("Power (MW)")
    plt.legend(loc="upper right")
    plt.grid(True, alpha=0.3)
    plt.tight_layout()
    plt.savefig(output_path)
    print(f"Saved water-filling plot: {output_path}")
    plt.close()

def plot_convergence(history, output_path="results/brd_convergence.png"):
    """Plots the convergence history of the BRD algorithm."""
    plt.figure(figsize=(8, 5))
    plt.plot(range(1, len(history) + 1), history, marker='o', color='#5e60ce',
        linewidth=2)
    plt.yscale('log')
    plt.xlabel('Iteration')
    plt.ylabel('Move Norm (Log Scale)')
    plt.title('BRD Algorithm Convergence History')
    plt.grid(True, which="both", ls="-", alpha=0.2)
    plt.tight_layout()
    plt.savefig(output_path)
    print(f"Saved convergence plot: {output_path}")
```

```
    plt.close()

def plot_co2_comparison(results, output_path="results/co2_comparison.png"):
    """Generates a bar chart comparing CO2 emissions across scenarios and policies."""
    scenarios = [f"{r['year']} {r['season'].capitalize()}" for r in results]
    pc = [r['pc_co2'] / 1e6 for r in results]
    op = [r['op_co2'] / 1e6 for r in results]
    brd = [r['brd_co2'] / 1e6 for r in results]

    x = np.arange(len(scenarios))
    width = 0.25
```